OpenAPI

OpenAPIとは

- OpenAPI Specification(OAS): HTTP API を記述する標準仕様(言語に依存しない)
- 仕様ファイルは YAML または JSON(推奨名は openapi.yaml か openapi.json)
- ドキュメント生成・コード生成・テストなどのツールで使える
- 策定は OpenAPI Initiative(OAI)、Linux Foundation 配下の団体
- OAI は、ベンダー中立な記述形式を掲げる
- もとは、SmartBear が寄贈した Swagger Specification
- 最新版は 3.2.1(2026年10月時点)、末尾の数字は修正・補足のみ

https://spec.openapis.org/oas/latest.html
https://www.openapis.org/about

仕様から型を作る

```yaml
openapi: 3.1.0
paths:
  /orders/{order_id}:
    get:
      ...
              schema: { $ref: "#/components/schemas/Order" }
components:
  schemas:
    Order:
      type: object
      required: [id, status]
      properties:
        id: { type: string }
        status: { type: string, enum: [pending, paid, shipped] }
```

https://spec.openapis.org/oas/latest.html
https://learn.openapis.org/best-practices

```
$ npm i -D openapi-typescript typescript
$ npx openapi-typescript ./openapi.yaml -o ./schema.d.ts
openapi-typescript 7.13.0
./openapi.yaml → ./schema.d.ts [233.6ms]
```

```
...
Order: {
    id: string;
    /** @enum {string} */
    status: "pending" | "paid" | "shipped";
};
...
```

- 実行結果は openapi-typescript 7.13.0(2026年10月時点)
- ツールの公式ドキュメントの対応版は OpenAPI 3.0 と 3.1(2026年10月時点)

https://openapi-ts.dev/introduction
https://openapi-ts.dev/cli

```ts
import type { components } from "./schema";

type Order = components["schemas"]["Order"];

export const isPaid = (order: Order) => order.status === "paid";
```

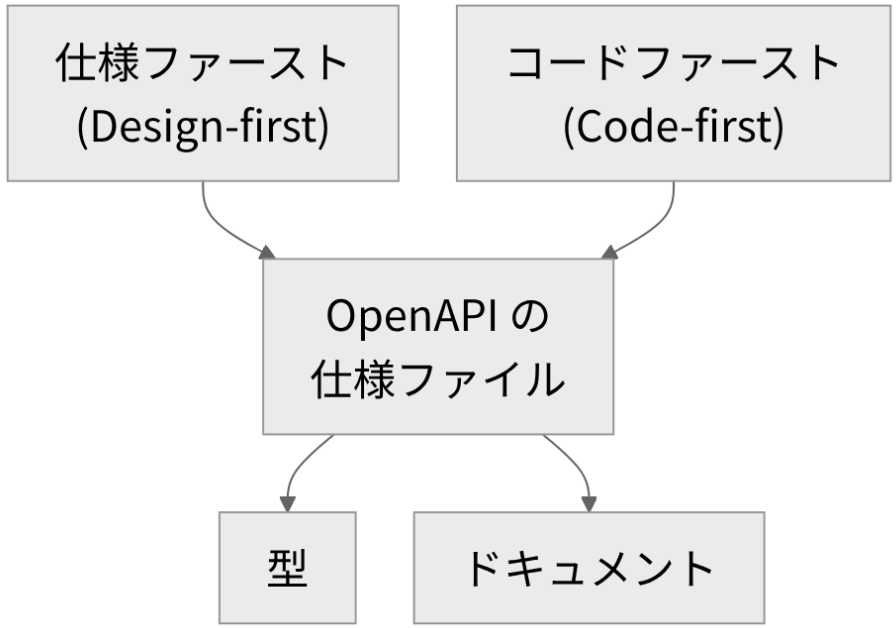
仕様の変更: status → state(型を再生成して、型チェックを実行)

```
$ npx tsc --noEmit
client.ts(5,47): error TS2339: Property 'status' does not exist
on type '{ id: string; state: "pending" | "paid" | "shipped"; }'.
```

【差し込み: 実務で型生成を使って、型のズレが減った(または困った)実体験】

https://openapi-ts.dev/introduction
https://www.typescriptlang.org/tsconfig/#noEmit

仕様ファーストとコードファースト



- OAI のガイドは仕様ファーストを推奨
- コードファーストの例: FastAPI は、書いたコードから仕様を自動で作る
- どちらの流れでも、型を作る手順は変わらない

https://learn.openapis.org/best-practices
https://fastapi.tiangolo.com/tutorial/first-steps/

まとめ

- OpenAPI は、HTTP API を記述する標準仕様
- 仕様を中心に置くと、型とドキュメントを機械的に作れる
- 仕様の変更は、型の再生成で型エラーとして見つかる(今回の例)
- OAI のガイドは仕様ファーストを推奨、型の生成はどちらの流れでも同じ